COMP1035 Software Engineering — Full Revision Notes

Lecture 01–18 · English key points for the exam, with Chinese study aids

Organised along the Revision-deck process map

Contents

How to use this — The **English** text is what you should memorise and write in the exam (answers must be in English). The light-blue boxes are 中文助记: short Chinese notes that only help you *understand and remember*, not what you write down. Every point is tagged with its source lecture, e.g. [Lec07].
Exam format (Revision deck): text answers, **no code writing**; you may be asked to **draw diagram(s)**; some **multiple-choice**; *write as much as you can*.

Course Map — the SE Process [Revision]

Software engineering turns "making software" into a **managed process**:

```
REQUIREMENTS                DEVELOPMENT              EVOLUTION
Elicitation  ->             Unit Testing  ->         Maintenance &
Modelling    ->             User Testing             continued change
Specification ->           (Git / Build / Deploy)
Validation
```

Two ideas run through the whole module:

- **Everything in SE exists to improve software quality.** [Lec15]
- **Change is inevitable** — requirements, environment and errors keep changing, so the process must cope with change. [Lec14]

中文助记：整门课就是把" 写软件" 变成一条可管理的流程：需求（获取-> 建模-> 规格-> 验证）-> 开发（含测试、Git、部署）-> 演化维护。背两条主线：(1) 所有流程都为提升质量；(2) 变化必然，流程要能应对变化。

1. Introduction: Software & Software Engineering [Lec01]

- **Software** is more than code — it is complex enough to *"need a big process"* to build. That complexity is exactly why we must **engineer** it.
- **Software Engineering (SE)** — a **systematic, engineering approach** to the development, operation and maintenance of software.
- **Software process** — the set of activities and their ordering that produce a software product. Typical **lifecycle phases**: *Requirements & Specification -> Design -> Implementation -> Testing -> Deployment -> Maintenance*.

中文助记：为什么要" 工程化"？因为软件常常质量差、超期、超预算（历史上有著名失败案例）。SE 引入过程 (process)、方法 (methods)、工具 (tools)，把不可控的" 写代码" 变成可管理、可度量、可重复的活动。"Software is complex enough to need a big process" 是关键一句。

2. Version Control & Git [Lec02] [Lec09] [Lec10]

> Mostly lab/practical; in the exam it appears as **concept questions**.

- **Version Control (VCS)** — keeps track of the **multiple versions** of system components, enabling rollback and team collaboration.
- Core Git workflow: **`add`** (stage) **`-> commit`** (save a snapshot into the repository — every commit is remembered) **`-> push/pull`** (sync with the remote) **`-> checkout`** (switch branch / restore an old commit).
- **Branch** — work on an independent copy; **`merge`** combines branches and may produce a **conflict** that must be resolved manually. [Lec09]
- **`.gitignore`** — list of file types Git ignores. **Tag** — a friendly name for a particular commit (e.g. a release version).
- **Git platforms** (GitLab / GitHub) [Lec10] — host repositories and add **issue tracking**, **Merge / Pull Requests** for code review, and **CI/CD** integration.

> 中文助记：分支工作流是难点——`feature branch -> merge request -> review -> 合并主分支`，不要直接改主分支，保证主分支随时可用。commit 记录每一次快照，可随时回退；merge 冲突要手动解决。

3. Requirements Engineering [Lec03]

- **Functional requirements** — *what the system should do* (specific services / behaviours).
- **Non-functional requirements** — **quality constraints** on the system (performance, security, reliability, usability…). Often describe *system qualities* rather than specific functions.
- **Stakeholders** — anyone with an interest in the system, in three layers:
  - **Primary** (direct users / "key stakeholders"),
  - **Secondary** (indirect users),
  - **Tertiary** (affected even more indirectly).
- **Elicitation techniques** — interviews, questionnaires/surveys, observation, document analysis, prototyping… **Have a strategy**: one method helps interpret another (e.g. observe an office first to decide *who* to interview).

Advanced technique: Ethnography

- Idea: *"there's no better way to learn than doing"* — an **anthropological**, observational method; the analyst **gets involved in the client's real work environment**. **Focused ethnography** targets specific tasks/roles.

Four benefits of ethnography (exam favourite, 2024 Q3):

1. Reveals **tacit / implicit knowledge** — practices people follow but cannot easily articulate.
2. Captures the **real process**, not the idealised "on-paper" one.
3. Reveals **cooperation and coordination** — the social/organisational side of work.
4. **Non-intrusive, low bias** — based on observation, not on what people *say* they do; finds requirements other techniques miss.

中文助记：功能需求 = 系统”做什么”；非功能需求 = 性能/安全/可靠性等”质量约束”。利益相关者分主/次/第三三层。民族志 =亲身融入客户环境观察，好处口诀：隐性知识 / 真实流程 / 协作配合 / 低偏差。

4. Requirements Elicitation: Scenarios & Stories [Lec04]

- **Scenario** — a written description of *how a user, in a context, achieves a goal*.
- **Components a scenario may include** (exam favourite, 2024 Q1):
  1. a **setting / context** (the environment and the things in it);
  2. one or more **actors / users** (often a **persona**);
  3. **goals / objectives** (what the user wants);
  4. a **plot** — the **sequence of events** (start to end), covering the **normal flow of events**, **what can go wrong and how it is handled**, and **other activities happening in parallel**.
- **User story** — an informal, short description of a feature *from the end-user's viewpoint* (“As a…, I want…, so that…”).
- **Persona** — a fictional but realistic character representing a class of users.

中文助记：用户故事是”一句话”粗粒度需求；场景是把它”展开成有细节的剧本”。场景四要素口诀：情境 / 参与者 / 目标 / 情节（情节里含正常流程、出错处理、并行活动）。

5. Requirements Modelling: UML [Lec05]

UML 2.5 has 13 diagram types; this module focuses on three behavioural ones (plus class diagrams):

Diagram	Shows	Key points
Use Case Diagram	the interactions between a **system and its environment**	high-level “what the system does”; **actor** + use case (oval) + system boundary
Activity Diagram	the **activities in a process / workflow**	usually **one activity diagram per use case** (not per use-case diagram); can use **swimlanes**
Sequence Diagram	interactions between actors/objects **over time**	lifelines, messages, combined fragments (e.g. **`alt`**)

Use case diagram relationships

- **Association** — a line linking an actor to a use case.
- **`«include»`** — a use case always uses the behaviour of another (reuse of common steps).
- **`«extend»`** — optional/conditional behaviour added to a use case (e.g. *Pay fine* extends *Return book* only when overdue).
- **Generalisation** — a specialised actor/use case inherits a general one (e.g. Student & Staff generalise to *Member*).

Use case vs activity diagram (2025 Q1b)

- Use case diagram = **breadth**: which functions exist and who uses them (“what”).

- Activity diagram = **depth**: expands **one use case** step by step ("how"), with sequence, decisions, parallelism, responsibilities.
- **Together** = complete understanding: scope (use case) + detailed behaviour (activity).

Activity diagram semantics (2023 Q3 difficulty)

- **Fork / Join** (bar) — a fork splits flow into **concurrent branches** (any order); a join waits for **all** incoming branches before continuing.
- **Decision** (diamond) — routes the flow along one edge by a **guard** (e.g. *failed* / *succeeded*).

> 中文助记：三种图记牢——用例图 = 做什么 (广度)、活动图 = 工作流 (深度,1 用例 1 图)、序列图 = 随时间的时序交互。«include» 必含、«extend» 可选扩展、generalisation 是继承。Fork/Join= 并发分支 (可换序，join 等全部到齐)；2023 Q3 那题正因如此，可行序列是 **d, e, h, i**。

Sequence diagram reading (2024 Q2)

Read messages top-to-bottom, sender-to-receiver, and turn each into a sentence; for an `alt` fragment, describe both the *success* and *failure* paths separately.

6. Requirements Specification [Lec06]

- **User requirements** — for **non-technical** stakeholders; describe only the system's **external behaviour**, *not* architecture/design; written in natural language with simple tables/diagrams.
- **System requirements** — more detailed; may use structured / graphical / mathematical notations.

Four notations for writing system specifications (exam favourite, 2023 Q1):

1. **Natural language** — plain-language sentences.
2. **Structured (natural language)** — standard **forms / templates**, tabulating requirements in an exact way.
3. **Graphical notations** — **UML** and other graphical models (e.g. sequence diagram).
4. **Mathematical specifications** — formal **mathematical models**; the most precise.

> 中文助记：四种记法口诀：自然语言 / 结构化 (表单模板) / 图形 (UML) / 数学 (形式化)，精确度依次升高。用户需求面向外行、只讲外部行为；系统需求更细。

7. Requirements Validation [Lec07]

Why validate (three reasons, 2025 Q2a):

1. **Checking that you are right** — *"the process of checking that requirements actually define the system that the customer really wants."*
2. **Avoiding rework** — *"errors in a requirements document can lead to extensive rework costs"*; fixing a requirements problem later (a system change) usually costs **more** than fixing design or coding errors.
3. **Contractually agreeing** — all stakeholders and the team **agree exactly what will be built**.

Systematic review checks — **Consistency**, **Completeness**, **Realism**, **Verifiability** (and: are requirements correct, necessary, important?).

Validation techniques (2025 Q2b):

1. **Requirements reviews** — a team systematically checks for errors and inconsistencies.
2. **Prototyping** — build an executable model; users/customers try it and give feedback.
3. **Test-case generation** — requirements should be testable; if a test is hard to design, the requirement is probably problematic.

> 中文助记：验证三理由——确认方向 (右) / 避免返工 / 契约共识。难点是返工成本曲线：错误发现越晚修复越贵（需求阶段改 vs 上线后改差几个数量级），所以验证性价比极高。四项检查：一致/完整/现实/可验证。

8. Prototyping [Lec08]

- **Prototype** — an early, executable model used to clarify requirements and explore design.
- **Two strategies**: **Throwaway** (built to learn, then discarded) vs **Evolutionary** (refined into the final product).
- **Fidelity**:
  - **Low-fidelity** — *"captures the point, the functions"*, used **to improve ideas**; e.g. sketches / paper prototypes / wireframes; early, cheap, fast.
  - **High-fidelity** — *"represents part of reality"*, used **to agree the final design** (final look & feel / functionality); any stage; more expensive.
- **Three purposes of a prototype**: (1) role of **technology**; (2) **look & feel**; (3) **implementation guide**.

When to use low vs high fidelity (2023 Q4):

- **Low**: early in development; to explore/compare/improve ideas cheaply and fast; when you only need to test "the point".
- **High**: to agree the final design; for realistic **usability testing** or a convincing **demo** close to the real product.

Prototyping risks (drawbacks):

1. Spending **too much time/effort on a high-fidelity** prototype when low-fidelity would do.
2. **Ad-hoc prototype code** (not professional standard) being **reused** in the real system.
3. Prototyping used **instead of**, rather than **alongside**, documentation.

> 中文助记：保真度——低保真(草图, 早期便宜, 改想法) vs 高保真(逼真, 定终稿, 做演示/可用性测试)。缺点口诀：高保真花太多 / 临时代码被复用 / 拿原型代替文档。

9. Test-Driven Development (TDD) [Lec11]

The Three Rules of TDD (Robert C. Martin — 2024 Q4, memorise the wording):

1. You are **not allowed to write any production code unless it is to make a failing unit test pass**.

2. You are **not allowed to write any more of a unit test than is sufficient to fail** (compilation failures count as failures).
3. You are **not allowed to write any more production code than is sufficient to pass the one failing unit test**.

- Cycle: **Red** (write a failing test) **-> Green** (write just enough code to pass) **-> Refactor**.
- Important: when you run a test, **check that it fails first** — otherwise it tests nothing.

Code coverage

- **Statement coverage** = *executed statements* / *total statements* — each statement run **at least once**.
- **Branch coverage** (node testing) — cover each branch.
- **(Compound) condition coverage** — test each boolean condition **true and false**.

Coverage calculation (2023 Q5 — identical to the slide example), for the `if(result>0)` snippet (7 statements):

Test	Path	Statement	Condition
a=3, b=9	result>0 TRUE	**5/7**	1/2 (TRUE only)
a=−3, b=−9	result<=0 FALSE	**6/7**	1/2 (FALSE only)
Both	—	**7/7 = 100%**	**2/2 = 100%**

> 中文助记：TDD 三规则务必背英文原话（失败测试驱动 / 测试够失败 / 代码够通过）；循环 **Red->Green->Refactor**。覆盖率：**100% 语句覆盖 ≠ 100% 分支/条件覆盖**；n 路 if/elif/else 至少要 n 个用例。

10. Release & Acceptance Testing [Lec12]

Four testing stages (2023 Q6):

Stage	Tests what	By whom / focus
Unit testing	individual components **in isolation**	developers; defects found early
Integration testing	combined units — their **interfaces & interactions**	developers/testers
System testing	the **whole integrated system** vs system requirements	testers; near-production
Acceptance testing	the system vs the **customer's needs** — accept or not	customer / end-users

- **Scope** grows unit -> integration -> system -> acceptance; **responsibility** moves developers -> testers -> customer.
- **Integration approaches**: **Big-bang**; **Top-down** (uses **stubs** for undeveloped lower modules); **Bottom-up** (uses **drivers** for undeveloped higher modules); **Hybrid/Sandwich** (both, with stubs and drivers).
- **Test pyramid**: ~70% unit, 20% integration, 10% end-to-end.

- **Release testing** — a form of system testing that decides if a version is good enough to release. **Alpha** (in-house) / **Beta** (real users) testing.

Functional / black-box testing (2024 Q5)

- **Functional (black-box) testing** — checks whether **features work as specified**, **without** knowledge of the internal code (e.g. can a word processor create/save/open/ delete a file, do cut/copy/font tools work).
- **Benefits**: (1) the tester needs **no knowledge of the internal code** — written from the spec, so non-developers can do it; (2) tests from the **user's perspective**, unbiased by implementation, catching missing/incorrect functionality.

> 中文助记：四阶段口诀——单元-> 集成-> 系统-> 验收（范围变大、责任从开发-> 测试-> 客户）。集成方式：top-down 用 **stub**、bottom-up 用 **driver**、hybrid 两者都用。黑盒测试 = 只看功能不看代码，好处 = 无需懂代码 + 用户视角。

11. Configuration Management & Deployment [Lec13]

Configuration Management (CM) — track and **control changes** in the software. Four sub-activities:

1. **Version control** — manage multiple versions of components.
2. **System building** — assemble components/data/libraries, then compile and link into an executable.
3. **Change management** — handle change requests.
4. **Release management** — prepare software for external release and track which versions have been released to customers.

- **Baseline** — a fixed, agreed snapshot/version.
- **CI/CD** — **Continuous Integration** (frequent merges + automated build/test) and **Continuous Delivery/Deployment** (automated release pipeline).

> 中文助记：CM 四件事——版本控制 / 系统构建 / 变更管理 / 发布管理，解决" 多人多版本多环境" 的混乱，保证任何时候都能重建出某个确定、可发布的版本。Baseline= 稳定快照。

12. Evolution & Maintenance [Lec14]

- **Change is inevitable** — new requirements emerge, the business environment changes, errors must be repaired, new equipment is added, performance/reliability must improve.
- **85–90% of the software budget** in large firms goes to **changing/evolving existing software**, not building new (rebuilding is risky, slower, costlier). *"Maintenance is normally the longest lifecycle phase."*
- **Types of maintenance**:
  - **Corrective** — fix errors not found earlier;
  - **Adaptive** — enhance services / adapt to new requirements & environment;
  - **Perfective** — improve the implementation of system units;

- (*Preventive* — proactively improve maintainability.)

Lehman's laws of program evolution (Lehman & Belady)

Law	Meaning
Continuing change	a program used in the real world must change or become less useful
Increasing complexity	structure grows more complex; resources needed to simplify it
Large program evolution	self-regulating; size, release interval, error counts stay ~invariant
Organisational stability	development rate ~constant, independent of resources
Conservation of familiarity	incremental change per release is ~constant
Continuing growth	functionality must keep growing to satisfy users
Declining quality	quality declines unless the system adapts to its environment
Feedback system	evolution is a multi-loop, multi-agent feedback system

> 中文助记：核心矛盾——为了"有用"必须持续变化 **(Continuing change)**，但每次改都让系统更复杂、质量更易下降 **(Increasing complexity / Declining quality)**，所以重构、文档、好架构是对抗"软件腐化"的手段。维护三类口诀：纠错 / 适应 / 完善。

13. Software Quality [Lec15]

- **Software quality** = **doing every aspect well** — not a single "stage" but a pervasive **culture**. *"All SE processes are designed to improve software quality."*
- **Quality Assurance (QA) team** does three things: **(1) planning for quality** (feeds the project plan); **(2)** quality methods/processes; **(3) checking for quality**.
- **Quality attributes** (mostly **non-functional**): **functionality**, **reliability / dependability**, **performance / efficiency**, **usability**, **maintainability**, **security**, **portability**…
- Quality can be **measured** with **metrics** (e.g. predictors of maintainability); **reviews/inspections** also check quality.

> 中文助记：重点——质量是"文化/过程"不是"阶段"，不能等到最后才"测一下"，要 pervasive 贯穿全程。QA 团队三件事：规划质量 / 保证质量 / 检查质量。质量属性 ≈ 非功能需求那一组。

14. Agile Methods [Lec16]

Agile principles (selected from the 12)

1. Highest priority: **satisfy the customer through early and continuous delivery**.
2. **Welcome changing requirements**, even late in development.
3. **Deliver working software frequently** (weeks, not months).

4. Business people and developers **work together daily**.
5. **Face-to-face conversation** is the most efficient communication.
6. **Working software is the primary measure of progress.**
7. **Simplicity** — maximise the work *not* done.

Waterfall vs Agile (2024 Q6)

- **Waterfall** — **sequential, plan-driven**; software developed in phases "resembling a waterfall"; once a phase is done you move on. Heavy, top-down, detailed up-front spec. Cons: **low flexibility**, **hard to manage change**, **hard to measure progress within a phase**, expensive/slow to fix late problems.
- **Agile** — **iterative & incremental**; small increments, **responds to change quickly**, **continuous customer involvement**, rapid feedback; lightweight, flexible.
- **Key contrast**: Waterfall fixes requirements up front and resists change; Agile **embraces change** via short iterations and frequent delivery.

Agile frameworks (2024 Q7) — all uphold the Agile principles

- **Scrum** — lightweight; time-boxed **sprints**, roles (**Scrum Master, Product Owner, Team**), artifacts (**Product/Sprint Backlog**). *Optimises time & delivery.*
- **Kanban** — visualise the workflow, limit work-in-progress. *Focus on speed of delivery.*
- **XP (eXtreme Programming)** — engineering practices (**pair programming, TDD, continuous integration**). *Focus on team effectiveness.*

Obstacles / cons of Agile (2024 Q8)

- Teams tend to **neglect documentation**.
- Weak initial architecture/design forces **frequent refactoring**.
- **Harder to practise** than waterfall — members must be well-versed in Agile.
- Time-boxing + re-prioritisation -> some features **miss the timeline** -> extra sprints & **cost**.
- Relies on an **involved customer** (hard long-term); important tasks can be forgotten; organisational change is slow.

> 中文助记：瀑布 = 顺序锁需求 (抗变) vs 敏捷 = 迭代拥抱变化。框架记忆：**Scrum->** 时间与交付、**Kanban->** 速度、**XP->** 团队效能。障碍口诀：轻文档 / 频繁重构 / 难实践 / 难按期 / 依赖客户参与。

15. Risk Management [Lec17]

Definitions (2023 Q7 — memorise the wording)

- **A risk is the probability of unwanted consequences** of an event and decision — the probability of **failing to meet expectations**.
- **An opportunity is the probability of exceeding expectations** (the opposite of a risk).
- **A risk is NOT a problem** — *"a risk is a potential problem over which we have some choices"* (a *potential future* problem we can still act on).

Risk-management workflow (2023 Q8) — iterative

Risk Identification -> Risk Analysis -> Risk Planning -> Risk Monitoring

1. **Identification** — list possible risks (checklists, past projects, team input).
2. **Analysis** — assess each risk's **probability and impact** and prioritise.
3. **Planning** — choose a **strategy** for each significant risk.
4. **Monitoring** — track risks and the effectiveness of strategies throughout, repeat.

Risk-control strategies (2023 Q9)

- **Avoidance** — actions to **reduce the chance of the risk happening**.
- **Minimisation** — actions to **reduce the impact** if it happens.
- **Contingency** — a plan for **what you will do/change if it happens**.
- (**Transfer** — pass the risk to another party, e.g. a subcontractor.)
- **Best strategy = Avoidance**: *"avoidance is always the best strategy, if possible"* — prevention beats reaction.

> 中文助记：背原话——风险 = 未达预期的概率；机会 = 超出预期的概率；风险 ≠ 问题（是" 我们仍有选择权的潜在问题"）。流程：识别-> 分析-> 规划-> 监控。三策略：规避 (降发生) / 最小化 (降影响) / 应急 (发生后怎么办)，规避最佳。对应 2025 Q5：离职-> 最小化、接口延期-> 应急、客户改 UI-> 规避。

16. Project Planning [Lec18]

- **Milestone** — the end point of an activity; every task should produce a **tangible output / deliverable**.
- Planning tools: **PERT (1958)** — tasks & dependencies; **Critical Path Method (1960)**; **Gantt chart (1910)** — adds time to tasks/dependencies; **staff allocation** charts.

PERT & critical path (2025 Q3)

- A **PERT chart** shows tasks and their dependencies (good for **dependencies and parallel tasks**).
- **Critical path** — list all paths through the chart; the **longest path (worst case) is the critical path** — *"the critical path is the bottleneck route."*
- It is the **most important** because it sets the **shortest possible project duration**; its activities have **zero slack/float**; **any delay on it delays the whole project**.

> 中文助记：求关键路径 = 把每条" 起点-> 终点" 路径时长相加，取最长那条。例：2025 Q3 中 A-B-C-E-F-G=30 为关键路径 (>28>27)。关键路径上零时差，延误即拖延全项目。

17. One-page Quick-Recall Sheet

Topic	Must remember (English)	Lec
Ethnography benefits ×4	tacit knowledge / real process / cooperation / low bias	03
Scenario components	setting / actors / goals / plot	04

Topic	Must remember (English)	Lec
Three UML diagrams	use case (what) / activity (workflow, 1 per use case) / sequence (time)	05
UC relationships	association / «include» / «extend» / generalisation	05
4 spec notations	natural language / structured / graphical / mathematical	06
Validation reasons	checking you're right / avoid rework / contractual agreement	07
Validation techniques	reviews / prototyping / test-case generation	07
Low vs high fidelity	low = explore ideas (cheap, early) / high = final design, demo	08
TDD 3 rules	failing test drives code; test just enough to fail; code just enough to pass	11
Coverage	executed/total; 5/7 & 6/7 -> together 100%	11
4 testing stages	unit -> integration -> system -> acceptance	12
Integration	stubs (top-down) / drivers (bottom-up) / hybrid	12
Maintenance types	corrective / adaptive / perfective	14
Lehman	continuing change + increasing complexity + declining quality	14
Quality	a culture, not a stage; all SE improves quality	15
Waterfall vs Agile	sequential & fixed vs iterative & embraces change	16
Agile frameworks	Scrum (time) / Kanban (speed) / XP (team)	16
Risk	risk = prob. of failing expectations; risk =/= problem	17
Risk flow / strategy	identify->analyse->plan->monitor; avoidance is best	17
Critical path	longest path; zero slack; sets project duration	18

Answering tips (Revision deck): write in English, **draw diagrams when asked** (label every actor, node, dependency and guard), answer the multiple-choice, and **write as much as you can**.